CAHIER DES CHARGES DU PROJET

Architecture Multi-Agents Autonomes pour l'Analyse Financière Temps Réel

Etudiants :

Irvin TEMFACK
Ségolène SIEWE

Tuteur : Ahmad BITAR

Table des matières

1- Introduction 3
2- Objectif 4
3- Public visé 4
4- Etat actuel des ressources et périmètre du projet 5
5- Description du système à réaliser 6
6- Bénéfices attendus et cadre pédagogique 7
7- Contraintes à prendre 7
8- Matériels et logiciels nécessaires à l'utilisation 8
9- Matériels et logiciels nécessaires à la réalisation 9
10-Méthodologie d'évaluation du produit 10
12- Coûts envisagés 11
13- Délais de réalisation 11
15- Maintenance 12
16- Droits d'auteur 13

1- Introduction

Dans le cadre du Projet Étudiant, ce projet mobilise une équipe de deux étudiants qui agiront à la fois comme développeurs, architectes logiciels et testeurs. Il est conçu pour produire un système fonctionnel à forte valeur technique et pédagogique, supervisé par le tuteur du PE qui veillera à ce que les objectifs de qualité soient respectés.

Ce projet consiste en la conception et le développement d'un système multi-agents d'aide à la décision en investissement piloté par l'intelligence artificielle. L'idée centrale est de combiner trois types d'analyse : une analyse sentimentale des investisseurs, une analyse macro-économique du marché, et une analyse technique de risque. Le tout est orchestré par un agent qui joue le rôle d'interface intelligente avec l'utilisateur, sous la forme d'un mini site web, développée avec Streamlit.

Le système est entièrement développé en Python et s'appuie sur LangGraph comme framework d'orchestration multi-agents. Pour les modèles de langage, nous avons opté pour une combinaison hybride : Ollama (modèles open-source locaux) pour les tâches simples et peu coûteuses, Claude 3.5 Haiku pour l'agent d'analyse fondamentale, et Claude 3.5 Sonnet pour l'agent Portfolio Manager chargé des décisions complexes. Les sources de données d'actualités retenues sont Google News API et Finnhub.

Le développement s'étale d'Avril 2026 au 15 juin 2026, avec une livraison du système opérationnel au 8 juin 2026.

2- Objectif

L'objectif principal est de concevoir et développer un système multi-agents d'analyse financière, capable de collecter des données de marché, de les analyser sous trois angles complémentaires, et de communiquer des recommandations claires à l'utilisateur via une interface conversationnelle. Ces recommandations doivent être dans l'optique d'optimiser la performance d'un portefeuille de 10 actifs préalablement choisi tout en tenant compte de certains paramètres liés au profil de l'utilisateur. Les objectifs clés sont les suivants :

• **Réaliser un cahier des charges** : Rédiger un cahier des charges détaillé servant de référence tout au long du projet, définissant les fonctionnalités, les contraintes et les jalons.

• **Concevoir une architecture multi-agents** : Mettre en place une architecture multi-agents modulaire avec cinq agents spécialisés orchestrés par LangGraph, chacun ayant un rôle bien défini et indépendant.

• **Mettre en place un pipeline NLP** : Intégrer un pipeline NLP hybride (FinBERT + LLM) pour l'analyse de sentiment : FinBERT en premier filtre pour trier des centaines d'actualités, puis un LLM pour analyser les 5 plus importantes en détail.

• **Optimiser les coûts LLM :** Exploiter une stratégie LLM hybride et économique : Ollama pour les tâches simples, Claude 3.5 Haiku pour le tri et la classification, Claude 3.5 Sonnet pour la prise de décision complexe.

• **Développer l'interface utilisateur** : Construire une interface utilisateur conversationnelle avec Streamlit, permettant à l'utilisateur d'interroger le système, de demander des conseils en matière d'investissement.

• **Valider par le backtesting** : Mettre en place un module de backtesting sur données historiques pour évaluer la pertinence des stratégies générées par le système.

3- Public visé

Le public visé est double. D'une part, l'équipe pédagogique et le jury d'évaluation, qui jugeront la qualité technique et la rigueur du projet. D'autre part, tout utilisateur souhaitant obtenir des recommandations financières assistées par l'IA sur les actifs suivis.

L'interface conversationnelle sera volontairement accessible : l'utilisateur peut poser une question en langage naturel et recevoir une réponse synthétique. Aucune expertise technique n'est requise pour utiliser l'interface.

Les 10 actifs analysés par le système devront être diversifiés et minutieusement choisi au préalable au moyen d'une démarche structurée.

4- Etat actuel des ressources et périmètre du projet

Pour ce projet, nous disposons des ressources suivantes :

• **Matériel disponible** : Chaque membre dispose d'un ordinateur personnel et des outils de développement standard (VS Code, Antigravity, terminal, navigateur web).

• **APIs financières disponibles** : Les APIs yfinance (actions/forex), Binance (crypto en temps réel) et Finnhub (actualités financières, plan gratuit) sont accessibles sans coût. Google News API sera également intégrée comme source d'actualités.

• **Infrastructure de collaboration** : Un dépôt GitHub commun est créé pour versionner le code, coordonner les contributions et suivre l'avancement.

• **Stratégie LLM définie** : La stratégie LLM est arrêtée : Ollama pour les tâches simples, Claude 3.5 Haiku via API Anthropic pour l'agent fondamental, Claude 3.5 Sonnet pour l'agent Portfolio Manager.

Le cadre LangGraph comme framework d'orchestration est également retenu. La Vector DB sera créée pour stocker et rechercher les actualités après vectorisation via LangChain. Ces choix techniques sont fixés dès le début du projet.

5- Description du système à réaliser

Cette section définit l'architecture, les agents, les fonctionnalités et les technologies du système multi-agents. Le système est développé en Python, orchestré par LangGraph, et s'appuie sur une base de données SQLite pour la persistance et une Vector DB pour les actualités.

Architecture globale – Liste des agents

Le système est structuré autour de cinq agents spécialisés, orchestrés par LangGraph :

Agent	Rôle	Technologie principale	LLM utilisé
Agent 1 : Collecte des données	Récupère les cours en temps réels et l'historique de prix	Yfinance, Binance, asyncio	Aucun (traitement pur)
Agent 2 : Analyste macro-économique (Contexte)	Lie l'actualité globale, les annonces des banques centrales et calcule un score exploitable par l'IA	Google News API + Finnhub, FinBERT, LangChain, Vector DB	Claude 3.5 Haiku (5 €/mois)
Agent 3 : Analyste sentimental	Analyse le sentiment des investisseurs concernant nos 10 choix d'actifs, calcule un score d'opinion	FinBERT, Vector DB	Claude 3.5 Haiku (5 €/mois)
Agent 4 : Analyste technique de risque	Calcul de la matrice de covariance, volatilité, des correlations,…	Pandas, numpy, pandas-ta	Ollama (local, 0 €)
Agent 5 : Portfolio manager +Interface	Synthétise les analyses des 4 agents, fais les calculs sur l'allocation de portefeuille(au moyen d'un modèle que l'on choisira), prend des décisions et communique avec l'utilisateur via chat Streamlit	LangGraph, Streamlit, pandas, numpy, pandas-ta	Claude 3.5 Sonnet (20 €/mois)

Module de backtesting

Un module de backtesting est intégré au système pour valider les stratégies sur des données historiques. Il permettra d'évaluer les recommandations à certains benchmark notamment le S&P500. Les résultats du backtesting seront intégrés au rapport final.

6- Bénéfices attendus et cadre pédagogique

Bénéfices pour les utilisateurs

- **Automatisation complète du pipeline :** Le système centralise l'ensemble du pipeline : collecte de données multi-sources, NLP, analyse technique, décision et communication, dans un flux automatisé et cohérent.
- **Accessibilité via l'interface chat :** L'interface conversationnelle rend le système accessible sans expertise technique : l'utilisateur interagit en langage naturel et obtient des analyses synthétiques.

Cadre pédagogique et bénéfices pour l'équipe de développement

- **Compétences techniques :** Python avancé : asynchronisme, APIs, gestion de données financières, allocation de portefeuille, NLP avec FinBERT, bases vectorielles.
- **Intelligence artificielle appliquée :** LangGraph, prompt engineering appliqué à la finance, stratégie hybride de LLMs (local + cloud).
- **Travail collaboratif :** Gestion d'un projet en équipe avec dépôt GitHub commun, séparation des tâches, versionning et coordination des livrables.
- **Vision système complète :** Conception d'un système complet end-to-end : de l'ingéstion de données à l'interface utilisateur, en passant par l'IA et la base de données.

7- Contraintes à prendre

Cette section présente les limitations ainsi que les exigences techniques, budgétaires et temporelles qui encadrent le développement du projet.

Contraintes techniques

- **Choix des actifs :** Les 10 actifs doivent avoir été suffisamment étudiés et compris. Il est également nécessaire de se questionner sur le market timing les concernant.
- **Langage et framework :** Le système est développé en Python (≥ 3.10) avec LangGraph comme framework d'orchestration.
- **Bases de données :** SQLite est retenu pour la persistance locale des décisions et performances. La Vector DB stocke les actualités vectorisées.

- **Interface utilisateur :** L'interface Streamlit doit être accessible via navigateur sans installation client. Le chat doit également pouvoir afficher les visualisations.

Contraintes de sécurité et de performance

- **Sécurité des clés API :** Les clés API (Anthropic, Binance, Finnhub, Google News) doivent être stockées exclusivement en variables d'environnement et ne jamais être commitées sur GitHub.
- **Performance :** Le pipeline FinBERT + LLM doit s'exécuter rapidement par actif analysé. La latence de réponse du chat doit être optimisée pour une requête standard.
- **Robustesse :** Le système doit gérer les pannes réseau et les échecs d'API (retry avec backoff exponentiel).

Contraintes budgétaires et temporelles

- **Budget LLM :** Le budget LLM est fixé à ~25 €/mois au maximum (5 € Haiku + 20 € Sonnet). Ollama est utilisé systématiquement pour les tâches simples afin de limiter les appels aux APIs payantes.
- **Délai de livraison :** La livraison opérationnelle est fixée au 8 juin 2026
- **Ressources humaines :** L'équipe travaille en parallèle de ses autres cours. La gestion du temps et la coordination constituent des contraintes majeures, adressées par un découpage clair des tâches entre les deux membres.

8- Matériels et logiciels nécessaires à l'utilisation

Le système est accessible via une interface web. Aucun matériel spécifique n'est requis de la part de l'utilisateur final :

- Ordinateur de bureau ou portable (PC ou Mac) avec connexion Internet stable
- Tablette ou smartphone pour la consultation du dashboard (interface responsive Streamlit)
- Un navigateur web moderne : Google Chrome, Mozilla Firefox, Microsoft Edge ou Safari (versions récentes)

Pour les membres de l'équipe souhaitant exécuter le système en local, un environnement Python 3.10+ et certaines dépendances seront nécessaires. Si Ollama est utilisé en local, un GPU est recommandé pour des performances optimales.

9- Matériels et logiciels nécessaires à la réalisation

Cette section détaille les outils et l'environnement de développement requis pour l'équipe.

Matériel nécessaire

Chaque membre de l'équipe de développement devra disposer des équipements suivants pour travailler efficacement :

- **Ordinateur :** Un ordinateur de bureau ou portable (PC ou Mac), idéalement avec GPU pour exécuter Ollama en local.

- **Connexion Internet :** Un ordinateur de bureau ou portable (PC ou Mac), idéalement avec GPU pour exécuter Ollama en local.

Logiciels et outils nécessaires

Outil	Nom / Version	Usage
Langage	Python ≥ 3.10	Développement de l'ensemble du système
Framework IA	LangGraph	Orchestration des 5 agents
LLM local	Ollama (Mistral)	Tâches simples, 0 €
LLM cloud	Claude 3.5 Haiku + Sonnet (Anthropic API)	Analyse fondamentale et décision
NLP spécialisé	FinBERT (Hugging Face)	Filtre sentiment financer sur actualités
Vector DB	ChromaDB	Stockage des actualités vectorisées
Vectorisation	LangChain	Extraction des news
Données marché	Yfinance + python-binance	Cours actions/forex + crypto temps réel
Actualités	Google News API + Finnhub	Sources d'actualités financières
Data science	Pandas, numpy, pandas-ta	Calculs techniques
Interface	Streamlit	Page web
Base de données	SQLite + aiosqlite + SQLAlchemy	Persistance des décisions
IDE	Visual Studio Code	Editeur principal avec extensions Python

Outil	Nom / Version	Usage
Versioning	Git + GitHub	Gestion du dépôt commun
Test APIs	Postman / Thunder Client	Test des requêtes LLM et news APIs

10-Méthodologie d'évaluation du produit

L'évaluation de la réussite du projet s'appuiera sur une approche en trois volets, combinant l'analyse de la documentation, la présentation du travail et la démonstration fonctionnelle du site.

Rapport écrit

- **Le cahier des charges** (ce document) : version finale intégrant les évolutions du projet.
- **La documentation technique** : architecture des agents, schéma de la Vector DB et de SQLite, choix technologiques justifiés.
- **Le rapport de performance** : résultats du backtesting.
- **Le bilan de projet :** défis rencontrés, solutions apportées et conformité aux délais.

Présentation orale

- Présenter l'architecture multi-agents
- Démontrer le système en direct
- Analyser les résultats du backtesting sur les actifs suivis.
- Répondre aux questions du jury sur les choix techniques et la gestion du projet.

Tests du Systèmes

1. **Tests unitaires :** chaque agent est testé individuellement avec des données mockées.
2. **Tests d'intégration** : le pipeline complet (collecte → NLP → analyse technique → décision → chat) est validé sur données réelles.
3. **Backtesting :** validation des stratégies sur données historiques.

12- Coûts envisagés

La stratégie budgétaire repose sur l'utilisation maximale de ressources gratuites et open-source, avec un coût récurrent limité aux seules APIs LLM payantes :

Service / Ressource	Type	Coût estimé	Statut
Python, pandas, numpy, pandas-ta, SQLite, FinBERT...	Bibliothèques open-source	0 €	Gratuit
Yfinance, Binance API (lecture seule)	APIs financières	0 €	Gratuit
Finnhub (plan gratuit)	Actualités financières	0 €	Gratuit
GitHub, VS Code, Streamlit	Outils développement	0 €	Gratuit
Ollama (Mistral)	LLM local open-source	0 €	Gratuit
ChromaDB	Vector DB	0 €	Gratuit
Claude 3.5 Haiku (agent fondamental)	Anthropic API	~5 € / mois	Budgétisé
Claude 3.5 Sonnet (agent portfolio manager)	Anthropic API	~20 € / mois	Budgétisé

Le coût total estimé est d'environ 25 € par mois pour les LLMs cloud, soit ~75 € sur les 3 mois de développement actif. Toutes les autres ressources sont gratuites.

13- Délais de réalisation

Le projet est structuré en 5 phases s'étalant d'avril à juin 2026

Phase	Durée	Activités principales
Phase 1 – Initialisation	1 semaine	Configuration LangGraph, création agents, choix LLM (API d'Anthropic), création de la Vector DB, dépôt GitHub
Phase 2 – Développement Data & Technique + choix des 10 actifs	3 semaines	Agent collecte (yfinance + Binance WebSockets), agent analyste technique (pandas-ta), base SQLite
Phase 3 – Développement IA & NLP	3 semaines	Pipeline FinBERT + LLM (agents 2 & 3), agent fondamental, agent sentimental, agent Portfolio Manager, intégration LangGraph
Phase 4 – Interface	2 semaines	Interface Streamlit, module backtesting, tests d'intégration

Phase	Durée	Activités principales
Phase 5 – Tests & Finalisation	1 semaines (8–15 juin)	Tests temps réel sur données, backtesting, documentation, rapport, présentation

Livrables

- Cahier des charges (ce document) : document complet et validé.
- Code source complet sur GitHub
- Système fonctionnel déployable localement ou sur Streamlit Community Cloud.
- Documentation technique : architecture, choix techniques, guide d'installation.
- Rapport d'analyse des performances : backtesting et tests.
- Présentation orale (slides) pour la soutenance.

Ce calendrier et ces livrables permettront de suivre l'avancement du projet et d'assurer une livraison finale conforme aux attentes.

15- Maintenance

Maintenance évolutive possible

- Ajout de nouveaux actifs financiers ou sources de données.
- Mise à jour des modèles LLM lors de la sortie de nouvelles versions Anthropic.
- Intégration de nouvelles stratégies d'analyse ou d'indicateurs techniques supplémentaires.

16- Droits d'auteur

Cette section définit la propriété intellectuelle et les droits d'utilisation relatifs au système et à son contenu.

- **Code source :** Le code source complet est notre propriété intellectuelle. Conformément au cadre pédagogique, il sera publié sur GitHub sous licence open source, permettant à d'autres de le consulter, l'utiliser et le modifier en respectant les droits des auteurs originaux.
- **Bibliothèques tierces :** Les bibliothèques Python tierces utilisées (LangGraph, FinBERT, pandas-ta, etc.) conservent leurs licences respectives. Leur usage dans ce projet est conforme à ces licences.
- **Données financières :** Les données financières collectées via yfinance, Binance et Finnhub sont soumises aux conditions d'utilisation de ces services. Elles ne seront pas redistribuées commercialement.
- **Droits pédagogiques :** Le tuteur et les responsables du PE auront un droit de lecture du code et du système à des fins exclusivement pédagogiques et non commerciales.

En résumé, notre projet se distingue par une architecture multi-agents innovante pensée pour l'efficacité, l'autonomie et la pertinence analytique. Grâce à la combinaison de cinq agents spécialisés : collecte de données en temps réel, analyse fondamentale des actualités économiques, analyse sentimentale des investisseurs, analyse technique des graphiques et gestionnaire de portefeuille. Le système offre une vision à 360° du marché, automatisée et cohérente. En s'appuyant sur un pipeline NLP hybride et une stratégie LLM économique, le système ne se contente pas de collecter des données : il les transforme en recommandations intelligentes, justifiées et actionnables. L'interface conversationnelle Streamlit, rend ces analyses accessibles à tout utilisateur en langage naturel, sans expertise technique requise. C'est une solution moderne, modulaire et adaptée aux enjeux de la finance algorithmique d'aujourd'hui. L'innovation ne réside pas dans un modèle unique et coûteux, mais dans la combinaison d'outils open-source, d'une orchestration multi-agents maîtrisée et d'une chaîne de développement complète construite avec un budget maîtrisé.